

Практическое занятие 22. Работа с контекстом и подключение стилей (CSS)

Цель занятия:

- Понять, как передавать данные в шаблоны через контекст.
 - Подключить и настроить CSS-стили для улучшения внешнего вида сайта.
-

Задачи занятия

1. Введение в контекст и статические файлы в Django
 2. Передача данных через контекст
 3. Использование контекста в шаблонах
 4. Подключение CSS-стилей
 5. Использование стилей в шаблонах
 6. Запуск и проверка приложения
 7. Выполнение практического упражнения. Улучшение внешнего вида сайта
 8. Выполнение дополнительных заданий. Добавление медиа-файлов, создание адаптивного дизайна
-

Задача 1: Введение в контекст и статические файлы в Django

1.1. Что такое контекст в Django?

- **Контекст** — это словарь данных, которые вы передаете из представления (view) в шаблон.
- Позволяет динамически отображать информацию на странице, основываясь на данных, полученных или обработанных в представлении.

1.2. Работа со статическими файлами

- **Статические файлы** — это файлы, которые не изменяются при работе приложения: CSS, JavaScript, изображения и т.д.
- Django предоставляет специальный механизм для работы со статическими файлами, позволяя легко подключать их в шаблонах.

Задача 2: Передача данных через контекст

2.1. Обновление представления `event_detail`

Шаги:

1. Откройте файл `events/views.py`.
2. Добавьте или обновите функцию `event_detail`:

```
# events/views.py

from django.shortcuts import render, get_object_or_404

events = [
    {'id': 1, 'title': 'Концерт классической музыки', 'date': '2023-11-10 19:00', 'description': 'Концерт с участием известных исполнителей.', 'location': 'Концертный зал', 'organizer': 'Иван Иванов', 'category': 'Музыка', 'comments': [
        {'user': 'Пользователь1', 'comment': 'Потрясающее мероприятие!', 'created_at': '2023-11-11 10:00'},
        {'user': 'Пользователь2', 'comment': 'Очень понравилось!', 'created_at': '2023-11-12 12:00'},
    ]},
    # Добавьте другие мероприятия, если необходимо
]

def event_detail(request, event_id):
    # Статические данные, так как модели не используются
    event = next((item for item in events if item['id'] == event_id), None)
    if event:
        comments = event.get('comments', [])
        return render(request, 'events/event_detail.html', {
            'event': event,
            'comments': comments
        })
    else:
        return render(request, 'events/event_not_found.html')
```

Пояснение:

- `event` : Текущее мероприятие, выбранное по `event_id`.
 - `comments` : Список комментариев для данного мероприятия.
 - Мы используем статические данные, так как модели не введены.
-

Задача 3: Использование контекста в шаблонах

3.1. Обновление шаблона `event_detail.html`

Шаги:

1. Откройте файл `events/templates/events/event_detail.html`.
2. Измените его содержимое следующим образом:

```
<!-- events/templates/events/event_detail.html -->

{% extends 'events/base.html' %}

{% block title %}{{ event.title }}{% endblock %}

{% block content %}
    <h1>{{ event.title }}</h1>
    <p><strong>Дата и время:</strong> {{ event.date }}</p>
    <p><strong>Место проведения:</strong> {{ event.location }}</p>
    <p><strong>Категория:</strong> {{ event.category }}</p>
    <p>{{ event.description }}</p>

    <h2>Отзывы</h2>
    {% if comments %}
        {% for comment in comments %}
            <div class="comment">
                <p><strong>{{ comment.user }}</strong> - <em>{{
comment.created_at }}</em></p>
                <p>{{ comment.comment }}</p>
            </div>
        {% endfor %}
    {% else %}
        <p>Нет отзывов о данном мероприятии.</p>
    {% endif %}
{% endblock %}
```

Пояснение:

- Используем переменные `event` и `comments`, переданные из представления.
- Отображаем информацию о мероприятии и список комментариев.

Задача 4: Подключение CSS-стилей

4.1. Создание директории для статических файлов

Шаги:

1. **Создайте директорию** `events/static/events/css/`.
 - Если директории `static` или `css` не существуют, создайте их.
2. **Структура должна быть следующей:**

```
your_project/
├─ events/
│   └─ static/
│       └─ events/
│           └─ css/
│               └─ styles.css
```

4.2. Дополнение файла `styles.css`

Дополните файл `styles.css` в `events/static/events/css/`, добавьте следующий код:

```
/* events/static/events/css/styles.css */

body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f4f4;
}

nav {
    background-color: #333;
    color: #fff;
    padding: 15px;
    text-align: center;
}

nav a {
    color: #fff;
    margin: 0 10px;
    text-decoration: none;
}

nav a:hover {
    text-decoration: underline;
}

.content {
    padding: 20px;
```

```

}

h1, h2 {
    color: #333;
}

ul {
    list-style-type: none;
    padding: 0;
}

li {
    background-color: #fff;
    margin-bottom: 10px;
    padding: 10px;
    border-radius: 5px;
    box-shadow: 0 0 5px rgba(0,0,0,0.1);
}

.comment {
    background-color: #e9e9e9;
    margin-bottom: 10px;
    padding: 10px;
    border-radius: 5px;
}

.pagination a {
    margin: 0 5px;
    text-decoration: none;
    color: #4CAF50;
}

.pagination span {
    margin: 0 5px;
}

```

4.3. Настройка статических файлов в `settings.py`

- Убедитесь, что в `your_project/settings.py` указаны правильные настройки:

```

# your_project/settings.py

STATIC_URL = '/static/'
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
STATICFILES_DIRS = [BASE_DIR / 'events/static']

```

4.4. Подключение CSS-стилей в `base.html`

Шаги:

1. Откройте файл `events/templates/events/base.html`.
2. В секции `<head>` убедитесь в подключении стилей:

```
<!-- events/templates/events/base.html -->

<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Система управления мероприятиями{% endblock %}
</title>
    {% load static %}
    <link rel="stylesheet" href="{% static 'events/css/styles.css' %}">
</head>
<body>
    <!-- Остальной код -->
</body>
</html>
```

Пояснение:

- `{% load static %}`: Загружает тег `static` для использования в шаблоне.
- `<link rel="stylesheet" href="{% static 'events/css/styles.css' %}">`: Подключает файл стилей.

Задача 5: Использование стилей в шаблонах

5.1. Обновление шаблона `event_list.html`

Шаги:

1. Откройте файл `events/templates/events/event_list.html`.
2. Измените его содержимое следующим образом:

```
<!-- events/templates/events/event_list.html -->

{% extends 'events/base.html' %}

{% block title %}Список мероприятий{% endblock %}

{% block content %}
    <h1>Мероприятия</h1>
    <ul>
```

```

        {% for event in events %}
            <li>
                <a href="{% url 'events:event_detail' event.id %}">{{
event.title }}</a> -
                {{ event.date }}
            </li>
        {% empty %}
            <li>Нет доступных мероприятий.</li>
        {% endfor %}
    </ul>
{% endblock %}

```

Пояснение:

- Используем класс `li` из стилей для оформления списка мероприятий.

5.2. Обновление других шаблонов

- **Убедитесь**, что в других шаблонах вы используете соответствующие теги и классы для оформления.

Пример для `home.html`:

```

<!-- events/templates/events/home.html -->

{% extends 'events/base.html' %}

{% block title %}Домашняя страница{% endblock %}

{% block content %}
    <h1>Добро пожаловать на сайт мероприятий!</h1>
    <p>Здесь вы найдете информацию о наших последних событиях.</p>

    <h2>Предстоящие мероприятия</h2>
    <ul>
        {% for event in events %}
            <li>
                <a href="{% url 'events:event_detail' event.id %}">{{
event.title }}</a> - {{ event.date }}
            </li>
        {% empty %}
            <li>Нет предстоящих мероприятий.</li>
        {% endfor %}
    </ul>
{% endblock %}

```

Задача 6: Запуск и проверка приложения

6.1. Запуск сервера разработки

```
python manage.py runserver
```

6.2. Проверка работы приложения

- Перейдите на главную страницу:
 - <http://127.0.0.1:8000/>
- Убедитесь, что стили применяются корректно.
- Перейдите на страницу деталей мероприятия и проверьте отображение комментариев и стилей.

6.3. Устранение возможных проблем

- Если стили не применяются:
 - Убедитесь, что путь к файлу стилей в `base.html` указан правильно.
 - Проверьте настройки `STATIC_URL` и `STATICFILES_DIRS` в `settings.py`.
 - Убедитесь, что у вас запущен сервер разработки и он обновлен до последних изменений.

Задача 7: Практическое упражнение: Улучшение внешнего вида сайта

Задание для студентов:

- Добавьте свои стили или измените существующие для улучшения внешнего вида сайта.
 - Измените цвета, шрифты, отступы и т.д.
 - Добавьте оформление для кнопок, ссылок и других элементов.
 - Сделайте сайт более привлекательным и удобным для пользователя.

Пример изменений:

- Добавьте эффекты при наведении на ссылки в навигации:

```
nav a:hover {  
    background-color: #555;  
    padding: 5px;
```



```
border-radius: 5px;
}
```

- Измените фон сайта:

```
body {
    background-color: #e0e0e0;
}
```

Дополнительное задание

1. Добавление медиа-файлов

Цель: Настроить работу с изображениями, добавив возможность отображения изображений мероприятий.

Шаги:

1. Создайте директорию для медиа-файлов:

```
your_project/
├─ events/
│   └─ media/
│       └─ events/
```

2. Добавьте изображения мероприятий в `events/media/events/`.
3. Настройте `settings.py` для работы с медиа-файлами:

```
# your_project/settings.py

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'events/media'
```

4. Настройте URL для медиа-файлов в `your_project/urls.py`:

```
# your_project/urls.py

from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    # ... ваши маршруты ...
```

```
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)
```

5. Обновите представления и шаблоны для отображения изображений:

- В представлении `event_detail` добавьте путь к изображению:

```
def event_detail(request, event_id):
    # ... ваш существующий код ...
    event = {
        # ... другие поля ...
        'image': f'events/event_{event_id}.jpg',
    }
    # ... остальной код ...
```

- В шаблоне `event_detail.html` отобразите изображение:

```
{% if event.image %}
    
{% endif %}
```

Пояснение:

- `MEDIA_URL` и `MEDIA_ROOT` : Настройки для работы с медиа-файлами.
- `static()` : Функция для обслуживания медиа-файлов в режиме разработки.

2. Создание адаптивного дизайна

Цель: Использовать CSS-фреймворк, например, Bootstrap, для улучшения адаптивности и внешнего вида сайта.

Шаги:

1. Подключите Bootstrap в `base.html` :

```
<!-- events/templates/events/base.html -->

<!DOCTYPE html>
<html>
<head>
    <!-- ... ваш существующий код ... -->
    <!-- Подключение Bootstrap CSS из CDN -->
```

```

    <link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.c
ss">
</head>
<body>
    <!-- ... ваш существующий код ... -->
    <!-- Подключение Bootstrap JS и зависимостей -->
    <script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"></script>
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@4.5.2/dist/js/bootstrap.bundle.m
in.js"></script>
</body>
</html>

```

2. Обновите шаблоны, используя классы Bootstrap:

- Пример для `event_list.html`:

```

<!-- events/templates/events/event_list.html -->

{% extends 'events/base.html' %}

{% block content %}
    <div class="container">
        <h1 class="mt-5">Мероприятия</h1>
        <div class="list-group">
            {% for event in events %}
                <a href="{% url 'events:event_detail' event.id %}"
class="list-group-item list-group-item-action">
                    <h5 class="mb-1">{{ event.title }}</h5>
                    <small>{{ event.date }}</small>
                </a>
            {% empty %}
                <p>Нет доступных мероприятий.</p>
            {% endfor %}
        </div>
    </div>
{% endblock %}

```

3. Удалите или измените ваши собственные стили в `styles.css`, чтобы не конфликтовать с Bootstrap.

Заключение

Поздравляем! Вы успешно:

- Поняли, как передавать данные в шаблоны через контекст.
 - Научились использовать контекст в шаблонах для отображения динамического содержимого.
 - Подключили и настроили CSS-стили для улучшения внешнего вида сайта.
 - Улучшили внешний вид сайта, используя собственные стили или CSS-фреймворки.
-

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Успехов в разработке!

Вопросы для обсуждения

- **Как контекст помогает в передаче данных из представления в шаблон?**
 - Позволяет передавать любые данные в виде словаря, которые затем можно использовать в шаблоне для динамического отображения информации.
 - **Почему важно использовать статические файлы в Django и как их правильно подключать?**
 - Статические файлы необходимы для оформления и функциональности сайта. Django предоставляет удобный механизм для их организации и подключения через тег `{% static %}`.
 - **Какие преимущества дает использование CSS-фреймворков, таких как Bootstrap?**
 - Ускоряет разработку, предоставляет готовые стили и компоненты, обеспечивает адаптивность и кроссбраузерную совместимость.
-

Полезные ресурсы

- [Документация Django: Работа со статическими файлами](#)
- [Документация Django: Шаблоны](#)
- [Bootstrap Documentation](#)
- [CSS Flexbox Guide](#)